

Cours MERN Stack – Groupe Décalé – Séance 1

Mon premier serveur Express & Architecture Modulaire (SoC : Router / Contrôleur)

Dr. Abdelweheb GUEDES – École Polytechnique de Sousse – 2026–2027

Objectif	Construire un serveur Express, le tester avec le navigateur et Postman, puis le restructurer immédiatement en architecture professionnelle découplée (SoC : Router et Contrôleur), prêt pour MongoDB.
Plan	1. Environnement 2. Client, serveur & HTTP 3. Atelier API Blog 4. Architecture SoC 5. Pratique 6. Récapitulatif
Validation	En direct avec l'enseignant en séance (validation des routes Postman et de l'architecture).

Comment travailler avec ce document. Vous avancez **par petites étapes concrètes** selon le cycle d'apprentissage actif : **Prédisez** (anticipez le comportement **avant** d'exécuter), **À vous** (écrivez ou modifiez le code), **Point de contrôle** (vérifiez le résultat). Les **Notions** sont introduites au moment précis où le code en a besoin. Dans ce parcours intensif, nous passons dès aujourd'hui à l'architecture modulaire utilisée en entreprise.

1 Préparer l'environnement

Installez avec les réglages par défaut :

- **Node.js**, version **LTS** (<https://nodejs.org/>) : runtime qui exécute notre serveur JavaScript.
- **VS Code** (<https://code.visualstudio.com/>) : éditeur de code.
- **Postman** (<https://www.postman.com/downloads/>) : pour tester nos routes HTTP.
- **Git** (<https://git-scm.com/>) : suivi des versions.

Ouvrez un terminal (PowerShell sous Windows) et vérifiez :

```
node -v
npm -v
```

Point de contrôle. Deux numéros de version s'affichent (ex : v20.x.x ou v22.x.x, et 10.x.x). **npm** installe les bibliothèques comme Express. Sous Windows, si PowerShell refuse les scripts, tapez : `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`.

2 Client, serveur et HTTP

2.1 Une conversation en deux temps

Sur le web, tout échange est une conversation entre un **client** et un **serveur**. Le client (navigateur, Postman, plus tard React) envoie une **requête**. Le serveur l'analyse, traite l'action demandée et renvoie une **réponse**. Le protocole qui fixe les règles est **HTTP**.

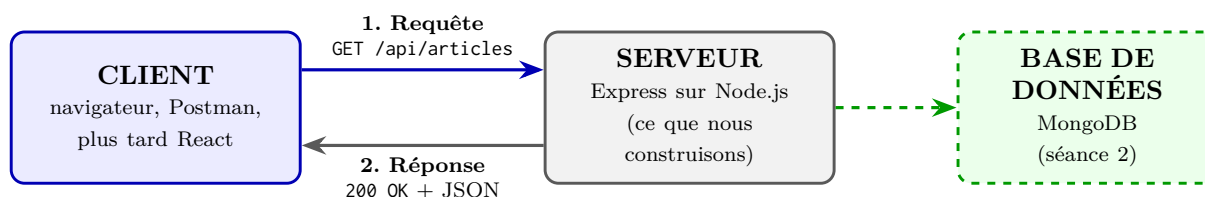


FIGURE 1 – Le client demande, le serveur répond. En séance 1, les données sont stockées en mémoire ; MongoDB viendra à la séance suivante.

Analogie : le restaurant. Vous (le client) passez commande au serveur (la requête). Le serveur revient avec le plat (la réponse). S'il manque un plat, le serveur l'indique poliment avec un code d'erreur au lieu de disparaître : c'est le rôle des **codes de statut HTTP**.

2.2 Une requête = une méthode + une URL

Une API **REST** structure ses URL autour des **ressources** (ici, les articles) et jamais autour d'actions :

Méthode	URL	Intention du client	Statut de succès
GET	/api/articles	« Donne-moi la liste de tous les articles. »	200 OK
GET	/api/articles/2	« Donne-moi l'article n°2. »	200 OK
POST	/api/articles	« Crée un article avec les données envoyées. »	201 Created
DELETE	/api/articles/2	« Supprime l'article n°2. »	200 OK / 204

Codes de statut à retenir : **2xx** tout s'est bien passé, **4xx** erreur côté client (400 données manquantes, 404 introuvable), **5xx** plantage du serveur.

2.3 Lire une URL

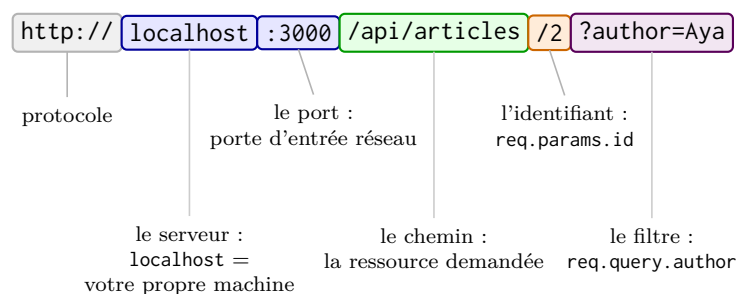


FIGURE 2 – Les morceaux d'une URL et l'endroit où Express les range.

Prédisez. Dans l'URL `http://localhost:3000/api/articles/2`, le 2 arrive-t-il dans le serveur sous forme de **nombre** ou de **texte** ? Notez votre réponse : vérification à l'étape 4.

3 TP guidé : l'API du blog (version initiale)

Étape 1 : créer le projet

Dans votre terminal :

```
mkdir mon-api-blog
cd mon-api-blog
npm init -y
npm install express
code .
```

Dans `package.json`, configurez la commande de démarrage avec rechargement automatique :

```
"scripts": {
  "dev": "node --watch server.js"
}
```

Désormais, `npm run dev` lance le serveur et le redémarre tout seul dès que vous enregistrez un fichier. Créez aussi un fichier `.gitignore` contenant la ligne `node_modules/`.

Étape 2 : un serveur qui dit bonjour

➡ Fichier `server.js` – lancer avec `npm run dev`

```
const express = require('express'); // 1. charger la bibliothèque Express

const app = express(); // 2. créer l'application serveur
const PORT = 3000;

// 3. route : quand un client demande GET /, exécuter cette fonction
app.get('/', (req, res) => {
  res.json({ message: "Bonjour, je suis l'API du blog" });
});

// 4. démarrer le serveur sur le port 3000
app.listen(PORT, () => {
  console.log('Serveur disponible sur http://localhost:${PORT}');
});
```

Notion : la fonction fléchée. `(req, res) => { ... }` est une **fonction fléchée** : syntaxe concise d'une fonction passée en argument. `req` représente la requête reçue, `res` l'objet servant à répondre.

Lancez `npm run dev`, puis ouvrez `http://localhost:3000` dans votre navigateur.

URL demandée : `http://localhost:3000`



FIGURE 3 – Le navigateur a envoyé GET / et affiche le JSON reçu.

Point de contrôle. Le terminal affiche `Serveur disponible...` et le navigateur affiche le JSON. Modifiez le message, enregistrez : le nouveau texte apparaît après rafraîchissement sans relancer le terminal.

Étape 3 : la liste des articles

Ajoutez le tableau de données **avant** `app.listen(...)` :

```
const articles = [
  { id: 1, title: 'Bienvenue sur le blog', author: 'Admin' },
  { id: 2, title: 'Mon premier serveur Express', author: 'Aya' },
  { id: 3, title: 'Tester une API avec Postman', author: 'Aya' }
];

// GET /api/articles -> tous les articles
app.get('/api/articles', (req, res) => {
  res.json({ total: articles.length, articles: articles });
});
```

URL demandée : <http://localhost:3000/api/articles>

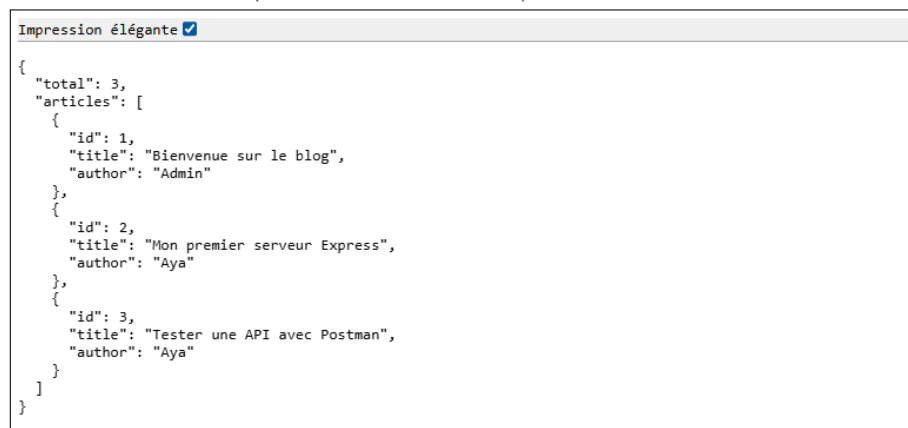


FIGURE 4 – Réponse de GET /api/articles : un objet JSON contenant le total et le tableau.

Point de contrôle. <http://localhost:3000/api/articles> affiche "total":3 et les 3 articles.

Étape 4 : récupérer un seul article

Le client demande GET /api/articles/2. Le 2 est dynamique (:id), Express le range dans req.params.id.

```

app.get('/api/articles/:id', (req, res) => {
  const id = Number(req.params.id);           // conversion obligatoire : "2" -> 2
  const article = articles.find(a => a.id === id);

  if (!article) {
    return res.status(404).json({ error: 'Article ${id} introuvable' });
  }
  res.json(article);
});

```

Notion : find. tableau.find(condition) parcourt le tableau et renvoie le **premier** élément satisfaisant la condition, ou undefined.

À vous. Test : Remplacez Number(req.params.id) par req.params.id et testez /api/articles/2. Que constatez-vous ? En JavaScript, "2" === 2 est faux ! Remettez ensuite Number(...).

URL demandée : <http://localhost:3000/api/articles/99>



FIGURE 5 – Pour un article inexistant, le serveur répond avec le statut 404 et un message JSON.

Point de contrôle. /api/articles/2 renvoie l'article d'Aya et /api/articles/99 renvoie l'erreur 404. Le return devant res.status(404) arrête l'exécution pour ne pas répondre deux fois.

Étape 5 : filtrer par auteur

Pour que GET /api/articles?author=Aya ne renvoie que les articles d'Aya, remplacez la route de l'étape 3 par :

```

app.get('/api/articles', (req, res) => {
  const { author } = req.query; // déstructuration : const author = req.query.author;

```

```
let resultat = articles;
if (author) {
  resultat = articles.filter(a => a.author === author);
}
res.json({ total: resultat.length, articles: resultat });
});
```

Notion : déstructuration et filter. `const { author } = req.query;` extrait directement la propriété. `tableau.filter(condition)` renvoie un **nouveau tableau** avec tous les éléments qui respectent le critère.

Étape 6 : créer un article avec POST

Pour créer un article, le client transmet des données dans le **corps (Body)** de la requête. Express a besoin d'un middleware pour lire ce corps JSON : placez-le **juste après** `const PORT = 3000;` :

```
app.use(express.json()); // parse le corps JSON et remplit req.body
```

Puis ajoutez la route avant `app.listen(...)` :

```
let prochainId = 4;

app.post('/api/articles', (req, res) => {
  const { title, author } = req.body;

  if (!title || !author) { // validation obligatoire
    return res.status(400).json({ error: "Le titre et l'auteur sont obligatoires" });
  }

  const nouvelArticle = { id: prochainId++, title, author };
  articles.push(nouvelArticle);
  res.status(201).json({ message: 'Article créé', article: nouvelArticle });
});
```

Étape 7 : tester avec Postman

Le navigateur ne sachant envoyer que des requêtes GET, nous utilisons Postman :



FIGURE 6 – Test Postman : ① méthode POST ; ② URL ; ③ onglet Body ; ④ raw + JSON ; ⑤ Send.

Prédisez. Avant d'envoyer, prédisez le statut attendu :

- POST /api/articles avec { "title": "Test", "author": "Aya" } → Statut prédit :
- POST /api/articles avec { "title": "Sans auteur" } → Statut prédit :

Point de contrôle. Le premier test répond 201 Created, le second répond 400 Bad Request. GET /api/articles affiche désormais 4 articles.

4 Architecture Professionnelle : Séparation des Préoccupations (SoC)

Étape 8 : Pourquoi séparer ? Les limites du serveur monolithique

À ce stade, notre fichier `server.js` mélange tout :

1. La configuration d'Express et du réseau ;
2. L'aiguillage des URL (les routes) ;
3. Le traitement des données et la logique métier.

Dans un vrai projet, avoir 50 routes dans un seul fichier crée du code illisible (« spaghetti ») et ingérable en équipe.

Le principe SoC (Separation of Concerns). On découpe le projet en 3 responsabilités :

- **server.js** : initialise Express, active `express.json()` et écoute le port. Il ne contient **aucune route métier**.
- **routes/articleRoutes.js** : associe chaque URL et méthode HTTP à une fonction précise (l'aiguilleur).
- **controllers/articleController.js** : contient les fonctions qui reçoivent (`req`, `res`), manipulent les données et renvoient le JSON (le **travailleur**).

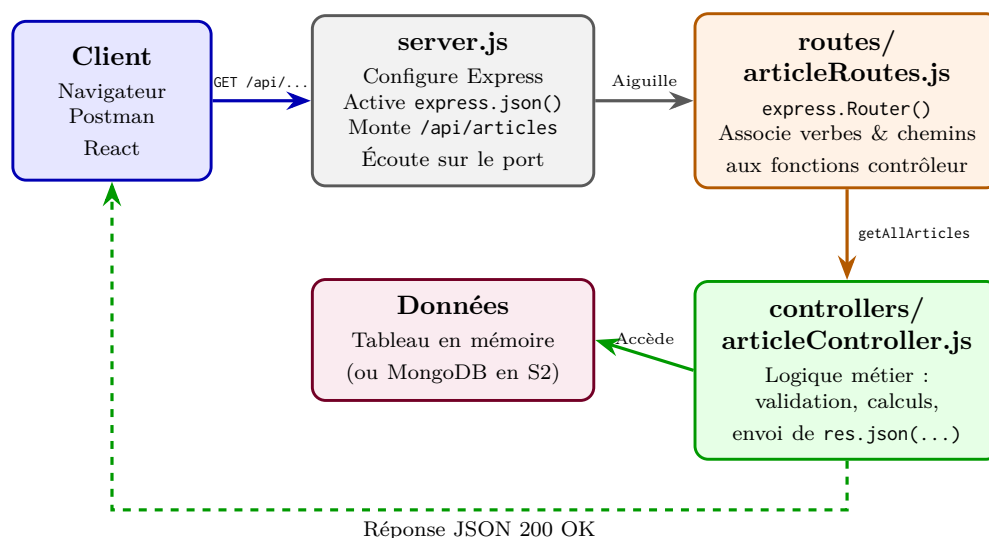


FIGURE 7 – Flux d'une requête dans l'architecture modulaire SoC.

Dans le terminal, créez immédiatement les deux dossiers d'architecture :

```
mkdir controllers routes
```

Étape 9 : Créer le contrôleur (`controllers/articleController.js`)

Créez le fichier et placez-y la gestion des données et les fonctions associées :

➡ Fichier `controllers/articleController.js` – lancer avec –

```
// Données isolées dans le contrôleur
let articles = [
  { id: 1, title: 'Bienvenue sur le blog', author: 'Admin' },
  { id: 2, title: 'Mon premier serveur Express', author: 'Aya' },
  { id: 3, title: 'Tester une API avec Postman', author: 'Aya' }
];
let prochainId = 4;

// 1. Récupérer tous les articles (avec filtre optionnel)
const getAllArticles = (req, res) => {
  const { author } = req.query;
  let resultat = articles;
  if (author) {
    resultat = articles.filter(a => a.author === author);
  }
  res.status(200).json({ total: resultat.length, articles: resultat });
};

// 2. Récupérer un article par son ID
const getArticleById = (req, res) => {
  const id = Number(req.params.id);
  const article = articles.find(a => a.id === id);
  if (!article) {
    return res.status(404).json({ error: 'Article ${id} introuvable' });
  }
  res.status(200).json(article);
};

// 3. Créer un nouvel article
const createArticle = (req, res) => {
  const { title, author } = req.body;
  if (!title || !author) {
    return res.status(400).json({ error: "Le titre et l'auteur sont obligatoires" });
  }
  const nouvelArticle = { id: prochainId++, title, author };
  articles.push(nouvelArticle);
  res.status(201).json({ message: 'Article créé', article: nouvelArticle });
};

// Exportation sous forme d'objet pour les rendre utilisables ailleurs
module.exports = {
  getAllArticles,
  getArticleById,
  createArticle
};
```

Notion : l'export CommonJS : `module.exports`. `module.exports = { funcA, funcB };` rend ces fonctions accessibles à d'autres fichiers via `require('./chemin')`.

Étape 10 : Créer le routeur (`routes/articleRoutes.js`)

Le routeur Express ne contient aucune logique : il fait uniquement la liaison entre une URL, une méthode et une fonction du contrôleur.

➡ Fichier `routes/articleRoutes.js` – lancer avec –

```
const express = require('express');
const router = express.Router(); // mini-application autonome de routage
```

```
const articleController = require('../controllers/articleController');

// Le préfixe /api/articles est défini dans server.js.
// Ici, '/' représente donc '/api/articles' !
router.get('/', articleController.getAllArticles);
router.get('/:id', articleController.getArticleById);
router.post('/', articleController.createArticle);

module.exports = router; // on exporte le routeur configuré
```

Notion : `express.Router()`. `express.Router()` crée un groupe de routes autonome. Remarquez que l'on écrit `router.get('/:id', ...)` au lieu de `app.get('/api/articles/:id', ...)` : le chemin devient court et réutilisable.

Étape 11 : Alléger `server.js` pour ne garder que l'essentiel

Nettoyez complètement `server.js`. Il ne conserve plus que la configuration globale :

➡ Fichier `server.js` – lancer avec `npm run dev`

```
const express = require('express');
const articleRoutes = require('./routes/articleRoutes');

const app = express();
const PORT = 3000;

// Middlewares obligatoires
app.use(express.json());

// Montage du routeur modulaire sous le préfixe /api/articles
app.use('/api/articles', articleRoutes);

// Route racine pour vérifier que le serveur tourne
app.get('/', (req, res) => {
  res.json({ message: "API du Blog - Serveur Modulaire Opérationnel" });
});

app.listen(PORT, () => {
  console.log('Serveur modulaire en écoute sur http://localhost:${PORT}');
});
```

Prédisez. Après ce découpage, si vous envoyez GET `http://localhost:3000/api/articles/2` sur Postman, la réponse change-t-elle pour le client ?

Point de contrôle. Re-testez sur Postman : **la réponse est strictement identique** ! Le client n'a pas vu de différence, mais votre code est maintenant ordonné, modulaire et maintenable.

5 À vous : modulariser le domaine users

Pour valider l'architecture SoC en autonomie avant la fin de la séance :

À vous. Mission en séance : Ajouter les utilisateurs en suivant le même découpage SoC

1. Créez `controllers/userController.js` :
 - Déclarez un tableau `users` avec 2 utilisateurs fictifs (`id`, `name`, `email`).
 - Écrivez `getAllUsers` (renvoie le tableau).
 - Écrivez `getUserById` (renvoie l'utilisateur ou 404).

- Écrivez `createUser` (vérifie `name` et `email`, renvoie 201).
- N'oubliez pas `module.exports = { ... };`.
- 2. Créez `routes/userRoutes.js` :
 - Importez `Express`, instanciez le routeur et importez `UserController`.
 - Déclarez `router.get('/', ...)`, `router.get('/:id', ...)` et `router.post('/', ...)`.
 - N'oubliez pas `module.exports = router;`.
- 3. Dans `server.js`, importez `userRoutes` et montez-le avec :

```
const userRoutes = require('./routes/userRoutes');
app.use('/api/users', userRoutes);
```

- 4. Ouvrez Postman et testez :
 - GET `http://localhost:3000/api/users`
 - POST `/api/users` avec le corps JSON :


```
{ "name": "Sami", "email": "sami@poly.tn" }
```

Point de contrôle. Les requêtes `/api/users` répondent correctement avec les statuts 200 et 201. Montrez votre terminal et vos requêtes Postman à l'enseignant pour validation.

6 Récapitulatif

Six idées fondamentales à retenir

1. Le web est une conversation : le **client** envoie une requête HTTP (verbe + URL, parfois corps JSON), le **serveur** renvoie une réponse (code statut + JSON).
2. Une API REST nomme ses endpoints d'après les **ressources** (`/api/articles`), jamais d'après des actions.
3. Les paramètres arrivent à trois endroits : `req.params` (dans l'URL), `req.query` (après `?`) et `req.body` (corps JSON grâce à `express.json()`).
4. L'**architecture SoC (Separation of Concerns)** sépare le code en 3 couches :
 - `server.js` : configuration globale et écoute port.
 - `routes/` : association chemin/verbe → fonction avec `express.Router()`.
 - `controllers/` : fonctions de traitement métier et réponses JSON.
5. Exporter un module s'écrit `module.exports = ...`, et l'importer se fait par `require('./chemin')`.
6. Les données d'URL sont du **texte** : utilisez toujours `Number(...)` avant de comparer avec un identifiant numérique.

Je veux...	J'écris...
Créer une route isolée	<code>router.get('/:id', controller.getById);</code> dans <code>routes/</code>
Monter un module de routes	<code>app.use('/api/articles', articleRoutes);</code> dans <code>server.js</code>
Exporter plusieurs fonctions	<code>module.exports = { funcA, funcB };</code> dans <code>controllers/</code>
Exporter un routeur	<code>module.exports = router;</code> à la fin du fichier de route
Lire le corps JSON	<code>app.use(express.json());</code> puis <code>const { title } = req.body;</code>
Lire un paramètre d'URL	<code>const id = Number(req.params.id);</code>
Lancer avec auto-reload	<code>npm run dev</code> (avec <code>node --watch server.js</code>)

Je sais faire (auto-évaluation)

☐	Expliquer ce qu'est une requête, une réponse et un statut HTTP (200, 201, 400, 404)	Section 2
☐	Créer un serveur Express avec <code>-watch</code> et lire du JSON avec <code>express.json()</code>	Section 3
☐	Lire <code>req.params</code> et <code>req.query</code> , utiliser <code>find</code> et <code>filter</code>	Section 3
☐	Découper une application selon l'architecture SoC (Router et Contrôleur)	Section 4
☐	Modulariser une nouvelle ressource en autonomie et la tester avec Postman	Section 5

La prochaine fois (Séance 2). Aujourd'hui, nos articles et utilisateurs sont stockés dans un tableau en mémoire qui s'efface à chaque redémarrage. Lors de la prochaine séance, nous brancherons une vraie base de données **MongoDB** avec **Mongoose**. Grâce à l'architecture SoC mise en place aujourd'hui, nous aurons uniquement besoin de modifier l'intérieur des **contrôleurs** (await `Article.find()`), sans toucher à nos fichiers de routes ni à `server.js`!